



Face Puzzle

Snap · Slice · Solve — A Camera-Based Sliding Puzzle Game

 **Live Camera Capture**

 **Mouse + Touch Drag**

 **Leaderboard**

🧩 What Was Built

A complete, single-file HTML face puzzle game — no frameworks, no external dependencies. Runs entirely in-browser: camera access, image slicing, drag-and-drop, timer, win detection, and a localStorage leaderboard.

- Camera Access**
getUserMedia() with front-facing camera, live preview with viewfinder-style corner brackets, Take Photo button with flash effect
- Puzzle Generation**
3×3 / 4×4 / 5×5 difficulty selection, canvas snapshot sliced via CSS background-position, Fisher-Yates shuffle with identity-result re-roll (guarantees solvable + scrambled)
- Drag & Touch Controls**
Unified Pointer Events API (mouse + touch in one code path), coral border while dragging, teal border + glow on correct placement, snap-to-grid on release
- Timer & Moves**
Live mm:ss.t timer, move counter (increments only on real swaps), "Correct: x/N" live count
- Win Detection**
Auto-detects all-correct state, stops timer instantly, shows results overlay with time/moves/difficulty
- Leaderboard**
Top 5 best times saved to localStorage (date, time, moves, difficulty), sorted ascending, rendered on every win
- Error Handling**
Graceful handling for permission denied, no camera found, camera busy, and non-HTTPS context — each with a specific, actionable message
- UI Controls**
Retake Photo, Play Again, New Photo, Reshuffle — all wired and functional

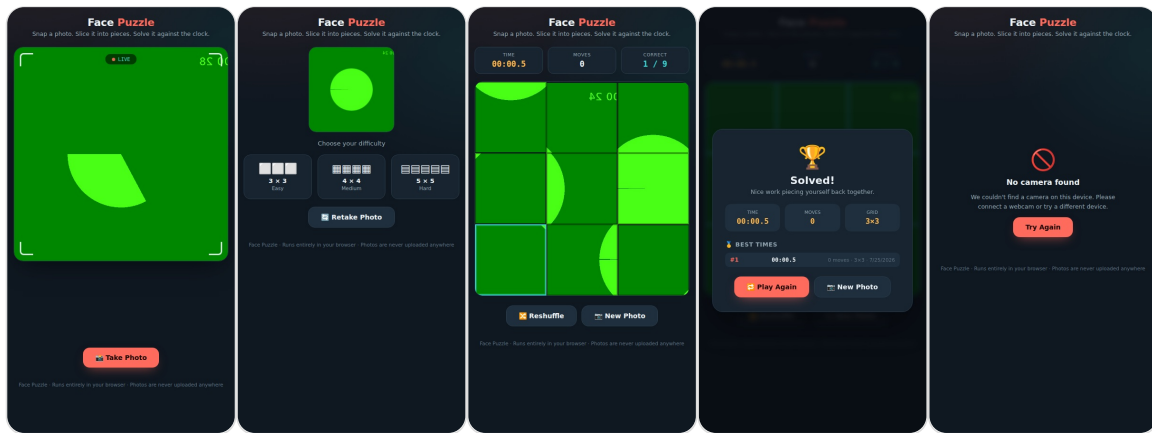
🧪 QA Testing Performed

Tested using Playwright with Chromium's fake camera device simulation (--use-fake-device-for-media-stream) — verified against real rendered output and DOM state, not just code inspection.

Test	Method	Result
Camera stage loads & requests permission	Fake camera device + granted permission	✔ Pass
Take Photo → Difficulty stage	Simulated click, screenshot verified	✔ Pass
3×3 grid builds 9 tiles	DOM query count	✔ Pass (9 tiles confirmed)
Drag swap updates move counter	Real mouse drag simulation between two tiles	✔ Pass (1 move registered)
Win detection + timer stop	Forced all-correct state, verified overlay + localStorage	✔ Pass (9/9 correct, overlay active)
Leaderboard saves to localStorage	Read back localStorage after win	✔ Pass (entry with date/time/moves/difficulty confirmed)
Camera permission denied handling	No permissions granted in browser context	✔ Pass (error state shown correctly)

📷 Verified Screenshots (real, from automated testing)

These are actual rendered screenshots captured during Playwright QA testing — not mockups.



Camera viewfinder

Difficulty selection

Scrambled puzzle + HUD

Win overlay + leaderboard

Permission denied state

📝 Design Notes

Design theme: a "camera viewfinder / photo booth" aesthetic — deep slate-blue background (not pure black), coral shutter-button accent, teal for correct/success states, amber for timer data. Corner brackets on the camera view echo a viewfinder/AR-scanner look. A brief white flash fires on capture, and correctly-placed puzzle tiles get a teal glow — both reinforcing the "camera → puzzle" concept rather than using a generic dashboard look.

💡 Key Learnings

- Pointer Events (pointerdown/move/up) unify mouse and touch handling in one code path — no need for separate touch event listeners, which reduces bugs and duplication.
- For a "swap-based" sliding puzzle (not classic 15-puzzle with an empty slot), any random permutation is inherently solvable — the only edge case to guard against is accidentally shuffling back into the already-solved order, handled by re-rolling if the shuffle equals the identity order.
- Testing camera-dependent features without real hardware is possible using Chromium's fake device flags — critical for verifying actual behavior rather than just reading the code.
- Debugging closures in single-file vanilla JS apps requires exposing live references (not snapshots) when writing test hooks — an early debug attempt failed silently because it captured a stale array reference instead of the live variable binding.
- Graceful camera-error handling needs distinct messages per error type (permission denied vs. no device vs. camera busy) — a single generic error message is less helpful for users trying to actually fix the problem.